LEADERS UNIVERSITY / LICENCE BI

Prompts, repères et dépannage

Atlas · une aide à ouvrir au bon moment

28 h	70 %	0
AU TOTAL	PRATIQUE	PRÉREQUIS IT

ATLAS PAPETERIE

Fichiers de ventes → ETL → Entrepôt → Rapport

03 / PROMPTS, REPÈRES ET AIDE

Youssef Douma · Leaders University

Édition pédagogique · 27 octobre 2026 · données fictives

AIDE / UN DOCUMENT PAR BESOIN

Trouver une aide au bon moment

Document	Quand l'utiliser
01_demarrer_windows_ubuntu.pdf	Installer le socle, ouvrir le dossier et choisir Docker ou le mode natif.
02_cours_atlas_ia.pdf	Suivre le parcours principal dans l'ordre : concepts, construction, essais, preuve.
03_prompts_reperes_depannage.pdf	Copier une demande, lire une instruction ou diagnostiquer une erreur.
04_defi_campus_box.pdf	Transférer la méthode à un cas avec statut et canal.
05_indices_et_corriges.pdf	Chercher un indice après une tentative ; consulter ensuite les corrigés.

Vous travaillez dans projet/. Le dossier solutions/atlas/ contient une référence séparée, pas votre point de départ. Le dossier outils/ fournit la reprise par jalon. Les fichiers CSV originaux restent dans projet/data/ ; les résultats sont produits dans projet/outputs/.

Le premier geste utile

Pour demander de l'aide, notez mission, fichier, commande, attendu et obtenu. « Ça ne marche pas » ne dit pas quelle expérience reproduire. Un petit cas avec une sortie réelle permet une correction ciblée.

ASSISTANCE / ACCÈS ET CONTEXTE

L'IA web constitue le parcours de base

Utiliser ChatGPT ou Claude dans le navigateur avec un compte personnel disponible. Le cours peut être suivi sans API, sans abonnement et sans outil agent. Les offres et limites peuvent varier : le nombre de demandes conseillé ici est une méthode de travail, pas une garantie du fournisseur.

1 Petit contrat → 2 Réponse de code → 3 Enregistrement → 4 Contrôle local

- Une demande vise une fonction ou un fichier court, avec entrée, sortie et exemple.
- Deux à quatre demandes ciblées par jalon sont une cible de sobriété : génération, correction si nécessaire, explication courte.
- Copier uniquement les colonnes utiles, quelques lignes fictives, le message d'erreur et la fonction concernée.
- Ne pas transmettre de mot de passe, clé, compte d'un pair ou données personnelles.

CLI facultative après vérification d'accès

Le livre 01 propose éventuellement une CLI sur le poste hôte. Une offre de chat gratuite ne prouve pas un droit à Claude Code ou à une CLI Codex. Vérifier l'éligibilité de votre compte avant toute installation et rester sur le web si elle n'est pas établie.

Si une limite apparaît, poursuivre les calculs, la lecture de code, les variations sur papier, les contrôles disponibles et les indices. Changer de compte pour contourner une limite n'est pas une étape du cours.

PROMPTS / SIX INFORMATIONS UTILES

Une demande qui laisse le contrôle au programmeur

À préciser	Exemple Atlas
But	Calculer le total du panier, pas tout le système BI.
Interface	calculer_total(ventes) -> int, nom inchangé.
Entrée	Liste de dictionnaires quantity / unit_price_millimes.
Sortie	Millimes entiers ; pas de input() ni effet de fichier.
Exemple	2×4500 + 3×1500 = 13500 ; liste vide = 0.
Périmètre	Remplacer seulement la fonction TODO ; garder le reste.

Un exemple n'est pas une constante à recopier dans la fonction. Le vérificateur utilise d'autres valeurs. Après réception, lire les opérations et vérifier que les arguments reçus sont réellement utilisés.

Pour corriger

« Mission 01, calcul.py. Attendu 13500, obtenu 6000. Voici ma fonction et les deux ventes. Explique l'opération manquante, puis corrige seulement cette fonction sans changer ses arguments. »

Ne fournissez pas tout l'historique de votre discussion. Si le contexte devient confus, créer ETAT_PROJET.md avec objectif, fichiers, dernier contrôle, erreur exacte et prochaine action. Cette note permet de reprendre proprement ailleurs.

REPÈRES / DEF, ARGUMENT, RETURN

Lire une fonction sans tout connaître de Python

Python · exemple autonome

```
def doubler(nombre):
    # Le bloc appartient à la fonction.
    resultat = nombre * 2
    return resultat

print(doubler(4))
```

def annonce la fonction ; nombre est le nom local de son entrée. Lors de doubler(4), nombre vaut 4. L'indentation de quatre espaces indique les instructions à exécuter dans la fonction. return rend la valeur à celui qui l'a appelée ; ici print écrit ensuite 8 à l'écran.

Signe	Lecture
=	Affecter une valeur à un nom : total = 0.
==	Comparer deux valeurs : quantité == 0.
["quantity"]	Lire la valeur liée à une clé de dictionnaire.
#	Commentaire pour le lecteur ; il n'est pas exécuté.

Créer docs/exemples/05_fonction.py et lancer sur une ligne depuis la racine : docker compose run --rm atelier python ../docs/exemples/05_fonction.py. En route native depuis projet/, utiliser py -3 ou python3 suivi du même chemin.

À ce stade, je sais...

montrer l'entrée, le bloc exécuté et la sortie d'une fonction ; expliquer la différence entre afficher une valeur et la renvoyer.

REPÈRES / FOR, IF, ELSE

Lire une boucle et une décision

Python · exemple autonome

```
quantites = [2, 0, 3]
total = 0
for quantite in quantites:
    if quantite > 0:
        total = total + quantite
    else:
        print("Quantité ignorée :", quantite)
print("Total :", total)
```

for passe sur les éléments d'une liste. if choisit un bloc si la condition est vraie ; else décrit l'autre cas. Cet exemple affiche la quantité ignorée 0 puis le total 5. Il illustre le contrôle de flux ; le vrai nettoyage doit en plus conserver la ligne rejetée avec son motif.

- Prédire le résultat avec [2, 1, 3] avant d'exécuter : total 6, aucune quantité ignorée.
- Observer les deux niveaux d'indentation : boucle puis décision.
- Repérer que print du total est hors de la boucle, donc n'apparaît qu'une fois.

Un import rend des outils disponibles, par exemple import csv. Un try / except sépare une tentative d'une erreur attendue. Un except qui ignore n'importe quelle erreur peut masquer un défaut : conserver le message utile et éviter les refus silencieux.

À ce stade, je sais...

suivre une petite boucle à la main et dire pourquoi une ligne entre dans une branche plutôt qu'une autre.

REPÈRES / FICHIER, DOSSIER ET TERMINAL

Les chemins : de quel dossier parle-t-on ?

Écriture	Sens
projet/calcul.py	Depuis la racine du cours, entrer dans projet puis ouvrir calcul.py.
Path(__file__).resolve().parent	Le dossier du fichier Python enregistré, même si le terminal part d'ailleurs.
ROOT / "data" / "produits.csv"	Construire un chemin avec pathlib sans dépendre des barres Windows ou Linux.
outputs/atlas.sqlite	Base produite par les missions 04 à 06.
outputs/rapport_atlas.html	Page produite depuis la base par la mission 07.

Avec Docker, les commandes partent toujours de la racine où se trouve compose.yaml ; le programme travaille dans /workspace/projet. En natif, placer le terminal dans projet/ pour les commandes python. Le livre 01 montre comment repérer ces deux niveaux.

Vérifier le dossier au lieu de deviner

Sous PowerShell, Get-Location affiche le dossier courant et Get-ChildItem ses fichiers. Sous Ubuntu, utiliser pwd puis ls. Si compose.yaml manque, vous n'êtes pas au niveau prévu pour docker compose.

Un nom de fichier est exact : ventes_ajout.csv et ventes-ajout.csv ne sont pas le même nom. Un chemin avec espaces se met entre guillemets dans le terminal. Ne pas coller un chemin personnel à l'intérieur des fonctions du projet.

REPÈRES / PETIT CONTRAT DU CODE

Les interfaces que les fichiers partagent

Fonction	Entrée → sortie
calculer_total(ventes)	Liste chiffrée → int en millimes.
lire_csv(chemin)	Chemin → liste de dict de textes.
nettoyer(lignes, produits, boutiques)	Lignes + 2 catalogues dict → (retenues, rejets, doublons).
creer_modele(chemin_base)	Chemin → création non destructive des tables ; retour None.
charger(ventes, produits, boutiques, chemin_base)	Retenues + catalogues + base → dict inserted/existing.
ajouter(chemin_source, chemin_base, chemin_data)	Chemins → dict raw/accepted/rejected/duplicates/inserted/existing.
generer(chemin_base, chemin_html)	Base → HTML + dict des indicateurs.

Les catalogues sont des dictionnaires : produits["P01"] vaut "Carnet" ; boutiques["NBE"] vaut "Nabeul". lecture.catalogues() et lecture.ecrire_csv() sont fournis ; les fonctions principales les réutilisent. Conserver les imports, helpers, main() et bloc de lancement des fichiers.

Formes attendues

Retenues et doublons : six champs source + unit_price_millimes + amount_millimes. quantity et les deux montants sont entiers. Rejets : six champs reçus + motif texte. Un renommage comme reason ferait perdre la raison au fichier CSV fourni.

REPÈRES / UNE SORTIE RÉUTILISABLE

Le dictionnaire du rapport

Clé de generer()	Valeur attendue
facts	Entier : nombre de faits.
quantity	Entier : nombre d'articles.
amount_millimes	Entier : montant calculé depuis la base.
period_start / period_end	Textes de dates ISO correspondant au min et max chargés.
shops / products	Deux listes de dictionnaires id, name, quantity, amount_millimes.

Le dictionnaire permet au programme d'assemblage et au vérificateur de réutiliser les chiffres. Le HTML est une autre présentation des mêmes mesures. Si ces sorties se contredisent, examiner ce qui a été recalculé ou écrit en dur.

Python · illustration autonome du formatage

```
def afficher_tnd(millimes):
    dinars = millimes // 1000
    reste = millimes % 1000
    return f"{dinars}.{reste:03d}"

print(afficher_tnd(115500))
```

Sortie : 115.500. // donne les dinars entiers ; % donne les millimes restants ; :03d affiche trois chiffres. Le helper formater_tnd() est déjà fourni dans rapport.py : ne le réécrivez pas pendant la construction de generer().

SQL / OBSERVER SANS MODIFIER

Exécuter une requête sur ma vraie base

Après la mission 05, créer docs/exemples/08_sql.py avec ce programme. Les chemins se fondent sur le script. Il ouvre la base existante en lecture seule et affiche les trois mesures globales. Avant l'ajout, attendre (12, 32, 99000) ; après ajout, (14, 35, 115500).

Python · à exécuter après le chargement

```
from contextlib import closing
import sqlite3
from pathlib import Path
racine = Path(__file__).resolve().parents[2]
base = racine / "projet" / "outputs" / "atlas.sqlite"
uri = base.as_uri() + "?mode=ro"
with closing(sqlite3.connect(uri, uri=True)) as connexion:
    resultat = connexion.execute(
        "SELECT COUNT(*), SUM(quantity), "
        "SUM(amount_millimes) FROM fact_sales"
    ).fetchone()
print(resultat)
```

Depuis la racine, saisir sur une ligne : docker compose run --rm atelier python ../docs/exemples/08_sql.py. En natif depuis projet/, utiliser py -3 ou python3 devant ../docs/exemples/08_sql.py.

Varier une seule question

Remplacer uniquement la requête par le SQL de docs/sql/boutiques.sql, en la plaçant entre trois guillemets si elle est sur plusieurs lignes. Remplacer fetchone() par fetchall() pour lire tous les groupes. Prédire les deux boutiques avant d'exécuter.

DEMANDES / À COPIER AU JALON CONCERNÉ

Demande 01 · à copier

Écris seulement la définition de calculer_total(ventes), en Python standard. Je remplacerai uniquement le bloc def/TODO de projet/calcul.py et conserverai main() et le bas du fichier. Fonction calculer_total(ventes) : ventes est une liste de dictionnaires contenant quantity et unit_price_millimes, deux entiers positifs. Renvoyer un entier égal à la somme quantity × unit_price_millimes, sans input(), sans afficher à la place du retour, sans lire ni écrire de fichier. Une liste vide renvoie 0. Exemple : 2 × 4500 et 3 × 1500 donne 13500 ; avec 3 carnets le total est 18000. Conserver exactement le nom et les arguments. Ajouter deux commentaires utiles, sans dépendance externe.

Avant d'envoyer

Joindre seulement le contrat et, si utile, l'emplacement TODO concerné avec ses imports. Aucun secret, aucune conversation complète, aucune copie du dépôt entier.

Après réception

- Lire l'entrée, la sortie et les opérations essentielles.
- Remplacer seulement la fonction annoncée ; garder les autres blocs fournis.
- Enregistrer puis lancer le module et son contrôle dans le même dossier de travail.
- Comparer la sortie réelle à votre prédiction ; expliquer une différence avant la demande suivante.

Le cours principal donne l'ordre et les valeurs de référence. Ces demandes sont des outils ; une réponse peut être différente de la référence et néanmoins correcte si elle respecte les mêmes comportements.

DEMANDES / À COPIER AU JALON CONCERNÉ

Demande 02 · à copier

Écris seulement la fonction lire_csv(chemin) -> liste de dictionnaires de textes. Les imports csv et Path et les chemins sont déjà fournis dans lecture.py ; conserver ROOT fondé sur __file__, catalogues(), ecrire_csv() et main(). Utilise csv.DictReader, séparateur virgule, UTF-8 avec prise en charge du BOM, newline="". Conserve toutes les lignes et colonnes ; ne nettoie, ne convertis et ne filtre rien. Le chemin est fourni par l'appelant. Un fichier absent doit produire une erreur claire. Aucun lancement de traitement à l'import. Le lot initial Atlas a 16 lignes hors en-tête. Explique en trois phrases pourquoi les valeurs restent str.

Avant d'envoyer

Joindre seulement le contrat et, si utile, l'emplacement TODO concerné avec ses imports. Aucun secret, aucune conversation complète, aucune copie du dépôt entier.

Après réception

- Lire l'entrée, la sortie et les opérations essentielles.
- Remplacer seulement la fonction annoncée ; garder les autres blocs fournis.
- Enregistrer puis lancer le module et son contrôle dans le même dossier de travail.
- Comparer la sortie réelle à votre prédiction ; expliquer une différence avant la demande suivante.

Le cours principal donne l'ordre et les valeurs de référence. Ces demandes sont des outils ; une réponse peut être différente de la référence et néanmoins correcte si elle respecte les mêmes comportements.

DEMANDES / À COPIER AU JALON CONCERNÉ

Demande 03 · à copier

Écris seulement la définition de nettoyer(lignes, produits, boutiques), en Python standard ; conserver les imports, constantes et main() fournis. Cette fonction renvoyant (retenues, rejets, doublons), trois listes. lignes : dictionnaires des six colonnes Atlas ; catalogues : dictionnaires identifiant vers libellé. Enlève les espaces, normalise les codes en majuscules. Vérifie identifiant présent, date ISO réelle exacte, codes connus, quantité entière positive, prix Decimal fini positif représentant un nombre entier de millimes. Les retenues gardent les six champs source normalisés ; quantity est entier, unit_price_tnd est un texte à trois décimales ; ajouter unit_price_millimes et amount_millimes entiers. Les rejets gardent les six champs source et une clé motif non vide ; les doublons ont les huit mêmes champs que les retenues. Garder les répétitions identiques à part. Même sale_id et contenu normalisé différent : lever ValueError explicite. Aucun traitement au moment de l'import. Le bilan initial doit être 12 retenues, 3 rejets, 1 doublon ; somme quantity 32 et somme amount_millimes 99000. Ne code pas ces résultats en dur.

Avant d'envoyer

Joindre seulement le contrat et, si utile, l'emplacement TODO concerné avec ses imports. Aucun secret, aucune conversation complète, aucune copie du dépôt entier.

Après réception

- Lire l'entrée, la sortie et les opérations essentielles.
- Remplacer seulement la fonction annoncée ; garder les autres blocs fournis.
- Enregistrer puis lancer le module et son contrôle dans le même dossier de travail.
- Comparer la sortie réelle à votre prédiction ; expliquer une différence avant la demande suivante.

Le cours principal donne l'ordre et les valeurs de référence. Ces demandes sont des outils ; une réponse peut être différente de la référence et néanmoins correcte si elle respecte les mêmes comportements.

DEMANDES / À COPIER AU JALON CONCERNÉ

Demande 04 · à copier

Écris seulement la définition de creer_modele(chemin_base), bibliothèque standard. Les imports sqlite3 et Path et connecter() sont déjà fournis et restent intacts. La fonction qui crée le dossier parent si besoin et quatre tables si absentes, sans supprimer une base existante. dim_product(product_key INTEGER PRIMARY KEY, product_id TEXT UNIQUE NOT NULL, product_name TEXT NOT NULL) ; dim_shop(shop_key INTEGER PRIMARY KEY, shop_id TEXT UNIQUE NOT NULL, shop_name TEXT NOT NULL) ; dim_date(date_key INTEGER PRIMARY KEY, full_date TEXT UNIQUE NOT NULL, year INTEGER, month INTEGER, day INTEGER) ; fact_sales(sale_id TEXT PRIMARY KEY, date_key INTEGER NOT NULL REFERENCES dim_date(date_key), product_key INTEGER NOT NULL REFERENCES dim_product(product_key), shop_key INTEGER NOT NULL REFERENCES dim_shop(shop_key), quantity INTEGER CHECK(quantity>0), unit_price_millimes INTEGER CHECK(unit_price_millimes>0), amount_millimes INTEGER CHECK(amount_millimes=quantity*unit_price_millimes)). Active PRAGMA foreign_keys=ON sur la connexion. Fermer proprement. Aucun INSERT et aucun traitement à l'import.

Avant d'envoyer

Joindre seulement le contrat et, si utile, l'emplacement TODO concerné avec ses imports. Aucun secret, aucune conversation complète, aucune copie du dépôt entier.

Après réception

- Lire l'entrée, la sortie et les opérations essentielles.
- Remplacer seulement la fonction annoncée ; garder les autres blocs fournis.
- Enregistrer puis lancer le module et son contrôle dans le même dossier de travail.
- Comparer la sortie réelle à votre prédiction ; expliquer une différence avant la demande suivante.

Le cours principal donne l'ordre et les valeurs de référence. Ces demandes sont des outils ; une réponse peut être différente de la référence et néanmoins correcte si elle respecte les mêmes comportements.

DEMANDES / À COPIER AU JALON CONCERNÉ

Demande 05 · à copier

Écris seulement la définition de charger(ventes, produits, boutiques, chemin_base) ; conserver les imports, connecter(), total_base(), lire_ventes_nettoyees() et main() fournis. La fonction renvoyant {"inserted": nombre, "existing": nombre}. ventes : retenues normalisées de nettoyer ; produits et boutiques : dictionnaires identifiant vers nom. La base a déjà été créée à la mission 04 ; si elle manque, lever FileNotFoundError. Utiliser connecter() fourni. Créer puis retrouver les dimensions par leurs codes ; date_key est AAAAMMJJ. Insérer les faits avec quantity, unit_price_millimes et amount_millimes. Active foreign_keys sur ta propre connexion. Un sale_id absent est inséré ; s'il existe avec date, produit, boutique, quantité et prix égaux, compter existing. S'il existe avec un contenu différent, lever ValueError et annuler toutes les écritures de ce lot. Une seule transaction de chargement ; pas de suppression, pas de INSERT OR REPLACE qui masque un conflit. Fermer proprement, aucun traitement à l'import. Base initiale : 12 faits, 32 articles, 99000 millimes ; second chargement identique : 0 insertion.

Avant d'envoyer

Joindre seulement le contrat et, si utile, l'emplacement TODO concerné avec ses imports. Aucun secret, aucune conversation complète, aucune copie du dépôt entier.

Après réception

- Lire l'entrée, la sortie et les opérations essentielles.
- Remplacer seulement la fonction annoncée ; garder les autres blocs fournis.
- Enregistrer puis lancer le module et son contrôle dans le même dossier de travail.
- Comparer la sortie réelle à votre prédiction ; expliquer une différence avant la demande suivante.

Le cours principal donne l'ordre et les valeurs de référence. Ces demandes sont des outils ; une réponse peut être différente de la référence et néanmoins correcte si elle respecte les mêmes comportements.

DEMANDES / À COPIER AU JALON CONCERNÉ

Demande 06 · à copier

Écris seulement la fonction ajouter(chemin_source, chemin_base, chemin_data), sans remplacer les imports ni main() fournis. Utiliser catalogues(chemin_data) et ecrire_csv() de lecture déjà importés. Utilise lire_csv de lecture, nettoyer de nettoyage et charger de chargement. Lis les catalogues du dossier chemin_data et transforme-les en dictionnaires identifiant vers nom. Le chemin_source désigne le lot à traiter ; aucun chemin personnel en dur. Nettoyer le lot, charger les retenues et renvoyer exactement un dictionnaire de six clés : raw, accepted, rejected, duplicates, inserted, existing, toutes entières. Écrire rejets_ajout.csv et doublons_ajout.csv à côté de la base avec ecrire_csv(), CHAMPS_SOURCE + ["motif"] et CHAMPS_VENTES déjà fournis. Réutiliser les règles existantes, ne pas créer une seconde version du validateur. Propager un conflit avec un message utile ; ne pas supprimer la base. Aucun traitement à l'import. Lot d'ajout : 3 reçues, 3 retenues, 0 rejet, 0 doublon interne, puis 2 inserted et 1 existing ; son rejeu : 0 inserted, 3 existing.

Avant d'envoyer

Joindre seulement le contrat et, si utile, l'emplacement TODO concerné avec ses imports. Aucun secret, aucune conversation complète, aucune copie du dépôt entier.

Après réception

- Lire l'entrée, la sortie et les opérations essentielles.
- Remplacer seulement la fonction annoncée ; garder les autres blocs fournis.
- Enregistrer puis lancer le module et son contrôle dans le même dossier de travail.
- Comparer la sortie réelle à votre prédiction ; expliquer une différence avant la demande suivante.

Le cours principal donne l'ordre et les valeurs de référence. Ces demandes sont des outils ; une réponse peut être différente de la référence et néanmoins correcte si elle respecte les mêmes comportements.

DEMANDES / À COPIER AU JALON CONCERNÉ

Demande 07a · à copier

Écris seulement la définition de generer(chemin_base, chemin_html) ; conserver les imports, SQL_BOUTIQUES, SQL_PRODUITS, connecter(), formater_tnd(), lignes_table() et main() fournis. La fonction qui interroge les quatre tables Atlas avec sqlite3, écrit un HTML autonome UTF-8 et renvoie exactement les clés facts, quantity, amount_millimes, period_start, period_end, shops, products. Les deux dernières sont des listes de dictionnaires id/name/quantity/amount_millimes ; les périodes sont des textes ISO. facts, quantity et amount_millimes sont entiers. Créer le dossier de sortie si besoin. Calculer depuis la base le nombre de faits, la quantité, le montant en millimes, les dates min/max et les montants par boutique et produit avec leurs libellés. Utiliser SUM, COUNT et JOIN avec les clés ; ne pas inscrire 115500 en dur. Afficher les TND avec trois décimales, tableaux lisibles, titre, période, unité, source et limites : ventes observées, sans stock/coût/marge/prévision. Échapper les libellés HTML, sans CDN, image distante, JavaScript externe ou accès réseau. Aucun traitement à l'import.

Avant d'envoyer

Joindre seulement le contrat et, si utile, l'emplacement TODO concerné avec ses imports. Aucun secret, aucune conversation complète, aucune copie du dépôt entier.

Après réception

- Lire l'entrée, la sortie et les opérations essentielles.
- Remplacer seulement la fonction annoncée ; garder les autres blocs fournis.
- Enregistrer puis lancer le module et son contrôle dans le même dossier de travail.
- Comparer la sortie réelle à votre prédiction ; expliquer une différence avant la demande suivante.

Le cours principal donne l'ordre et les valeurs de référence. Ces demandes sont des outils ; une réponse peut être différente de la référence et néanmoins correcte si elle respecte les mêmes comportements.

DEMANDES / À COPIER AU JALON CONCERNÉ

Demande 07b · expliquer un assemblage

Voici les appels de executer() déjà fourni. Explique en cinq phrases maximum leur ordre et la raison du deuxième appel à ajouter. Ne modifie aucun fichier et ne propose aucun nouveau helper. Je contrôlerai moi-même 14 faits, 35 articles et 115500 millimes après relance.

Avant d'envoyer

Joindre seulement le contrat et, si utile, l'emplacement TODO concerné avec ses imports. Aucun secret, aucune conversation complète, aucune copie du dépôt entier.

Après l'explication

- Comparer les phrases à l'ordre réel des appels de executer().
- Expliquer à un pair pourquoi ajouter est appelé deux fois.
- Conserver executer.py fourni sans le réécrire.
- Lancer executer.py puis verifier.py 07 et lire les bilans réels.

Le cours principal donne l'ordre et les valeurs de référence. Ces demandes sont des outils ; une réponse peut être différente de la référence et néanmoins correcte si elle respecte les mêmes comportements.

DÉPANNAGE / LIRE LA DERNIÈRE LIGNE

Mon script affiche une erreur

Symptôme	Action utile
Le terminal attend après >>>	Vous êtes dans la console Python ; sortir avec exit(), puis saisir la commande au terminal système.
Le fichier se termine par .txt	Afficher les extensions Windows puis enregistrer le vrai nom .py.
SyntaxError	Enlever les marques ``` et contrôler guillemets/deux-points à la ligne indiquée.
IndentationError	Observer les espaces du bloc def / for / if ; ne pas mélanger les niveaux.
ModuleNotFoundError local	Vérifier les noms des fichiers et le dossier projet ; ne pas installer sur internet un module du cours.
NotImplementedError	La fonction du jalon contient encore son TODO.
Sortie absente	Exécuter le module avant verifier.py ; le vérificateur ne crée pas vos résultats à votre place.

Demander une correction limitée

Copier le dernier message exact, la fonction concernée et une entrée minimale. Donner attendu et obtenu. Demander une seule cause, une correction locale et le test à relancer.

DÉPANNAGE / TROIS NIVEAUX À SÉPARER

La commande, la sortie et le sens

Ce qui réussit	Ce qui reste à contrôler
Docker lance Python	Le bon script a-t-il été enregistré ?
Le script termine	Les comptes et sommes correspondent-ils aux données ?
Le total est correct	Chaque ligne a-t-elle une issue et les clés sont-elles cohérentes ?
Le HTML est visible	Est-il à jour, issu de la base et honnête sur les limites ?
Le rejeu ne plante pas	L'état de la base est-il réellement inchangé ?

Pour les missions 03 à 07, l'ordre normal est : enregistrer la fonction → exécuter le module → lire les sorties → exécuter verifier.py NN. Les vérifications temporaires servent à éprouver vos fonctions ; les fichiers de votre projet servent à vérifier votre exécution réelle.

À ce stade, je sais...

décrire un blocage de manière reproductible, conserver les preuves utiles et demander une correction qui ne change pas tout le projet.

Références primaires des outils employés : https://docs.python.org/3/library/csv.html ; https://docs.python.org/3/library/decimal.html ; https://docs.python.org/3/library/sqlite3.html ; https://www.sqlite.org/lang_select.html. Les références d'installation et d'accès aux assistants sont rassemblées dans le livre 01.